3.7、Canny边缘检测

[OpenCV: Canny Edge Detection]
https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

Canny 算法是计算机视觉中最经典、最有效的边缘检测方法之一，由 John F. Canny 在 1986 年提出。它能很好地检测出图像中的强边缘，同时抑制噪声，得到“细而干净”的边缘图。

Canny 算法的四个主要步骤（Theory）

1. 噪声抑制 (Noise Reduction)
   边缘检测对噪声很敏感，所以第一步用 5x5 高斯滤波器 对图像进行平滑处理，减少噪声影响。
2. 计算强度梯度 (Finding Intensity Gradient)
   对平滑后的图像，使用 Sobel 算子 在水平（Gx）和垂直（Gy）方向计算梯度。
   然后计算每个像素的梯度幅度（Edge Gradient）和方向（Angle）：

   $$G = \sqrt{G_x^2 + G_y^2} \quad \theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

   梯度方向会被近似为 0°、45°、90°、135° 这四个方向之一（垂直、水平、两个对角线）。
3. 非极大值抑制 (Non-maximum Suppression)
   这一步让边缘变得“细”（thin edges）。
   沿着梯度方向检查每个像素，如果它不是邻域内的局部最大值，就把它的梯度值设为 0。
   这样只有**真正的边缘像素会被保留**下来，形成细线。
4. 双阈值滞后处理 (Hysteresis Thresholding)
   使用两个阈值：minVal（低阈值）和 maxVal（高阈值）。
   - 大于 maxVal 的像素 → 一定是边缘（sure-edge）。
   - 小于 minVal 的像素 → 一定不是边缘，直接丢弃。
   - 介于两者之间的像素 → 如果它和“sure-edge”相连，就认为是边缘；否则丢弃。
     这个步骤能**连接弱边缘（如果它和强边缘相连），同时去除孤立的弱噪声点。**

最终输出是一张二值图像：边缘像素为白色（255），非边缘为黑色（0）。
OpenCV 中的 cv.Canny() 函数OpenCV 把上面所有步骤封装成了一个函数，非常方便：

```
edges = cv.Canny(image, threshold1, threshold2, apertureSize=3,
L2gradient=False)
```

参数说明：

- image：输入图像（建议是灰度图）

- threshold1：minVal（低阈值）
- threshold2：maxVal（高阈值），通常 threshold2 ≈ 2~3 × threshold1
- apertureSize：Sobel 核的大小，默认 3（可以是 3、5、7）
- L2gradient：是否使用更精确的梯度计算公式（$\sqrt{(G_x^2 + G_y^2)}$）。默认为 False（用 |Gx| + |Gy|，更快但略不精确）。设为 True 时更准确。

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

# 1. 读取图像（灰度模式）
img = cv.imread('messi5.jpg', cv.IMREAD_GRAYSCALE)
if img is None:
    print("图片读取失败，请检查路径")
    exit()

# 2. 应用 Canny 边缘检测
edges = cv.Canny(img, 100, 200)   # minVal=100, maxVal=200

# 3. 显示结果
plt.subplot(121), plt.imshow(img, cmap='gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])

plt.subplot(122), plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Image'), plt.xticks([]), plt.yticks([])

plt.show()
```

- 阈值选择很重要：
  - 阈值太低 → 边缘太多，噪声也多。
  - 阈值太高 → 边缘太少，可能丢失重要细节。
  - 推荐从 100, 200 或 50, 150 开始尝试，根据图像调整。